

Apostila Completa — Inserção, Carga e Integração de Dados no SQL Server

Módulo 02 — Banco de Dados Relacionais

Este documento reúne, em um único arquivo, todo o conteúdo do diretório `02_DML_alteracao_insercao`, organizado na mesma sequência numérica dos arquivos originais, seguido dos roteiros de exercícios práticos.

Parte 01 — 2.1 Introdução

2.1 Introdução

A inserção de dados corresponde às operações da linguagem DML (Data Manipulation Language) responsáveis por adicionar registros às tabelas.

Objetivos

- Compreender os mecanismos de inserção.
- Selecionar o método adequado para cada cenário.
- Avaliar desempenho e manutenção.

Cenários comuns

- Cadastro de usuários.
- Importação de planilhas.
- ETL e Data Warehouse.
- Integração entre sistemas.

Métodos estudados

- INSERT ... VALUES
- INSERT múltiplas linhas
- INSERT ... SELECT
- SELECT INTO
- MERGE
- BULK INSERT
- SSIS
- bcp
- SqlBulkCopy

Boa prática: escolha o método considerando volume de dados, origem e frequência da carga.

Parte 02 — 2.2 INSERT ... VALUES

2.2 INSERT ... VALUES

Conceito

É o método mais simples para inserir registros. Os valores são informados explicitamente na instrução SQL.

Sintaxe

```
INSERT INTO Aluno (Matricula, Nome, Curso)
VALUES (1001, 'Maria Silva', 'Computação');
```

Explicação

- **INSERT INTO**: define a tabela.
- Lista de colunas: evita erros quando a estrutura mudar.
- **VALUES**: contém os dados.

Resultado esperado

```
(1 row affected)
```

Exemplo de projeto

Em um sistema acadêmico, quando um aluno conclui sua matrícula pela interface web, a aplicação executa um **INSERT ... VALUES** para gravar o cadastro.

Vantagens

- Simples.
- Legível.
- Ideal para transações OLTP.

Limitações

- Pouco eficiente para grandes volumes.

Exercício

Cadastre cinco professores utilizando **INSERT ... VALUES**.

Parte 03 — 2.3 INSERT com múltiplas linhas

2.3 INSERT com múltiplas linhas

Conceito

Permite inserir vários registros em um único comando.

```
INSERT INTO Curso (IdCurso, Nome)
VALUES
(1, 'Computação'),
(2, 'Engenharia'),
(3, 'Matemática');
```

Resultado esperado

(3 rows affected)

Aplicação

Ideal para carga inicial de tabelas de domínio, como estados, cidades, cursos ou perfis de usuários.

Comparação

- Mais eficiente que executar vários **INSERT** individuais.
- Não substitui BULK INSERT para grandes volumes.

Exercício

Cadastre dez disciplinas em um único comando.

Parte 04 — 2.4 INSERT ... SELECT

2.4 INSERT ... SELECT

Conceito

Insere registros utilizando o resultado de uma consulta.

```
INSERT INTO AlunoBackup (Matricula, Nome, Curso)
SELECT Matricula, Nome, Curso
FROM Aluno
WHERE Curso='Computação';
```

Funcionamento

1. O **SELECT** recupera os dados.
2. O SQL Server valida tipos e restrições.
3. Os registros são inseridos na tabela destino.

Resultado esperado

```
(120 rows affected)
```

Estudo de caso

Uma universidade mantém uma tabela histórica de alunos formados. Ao final de cada semestre, um processo copia os alunos concluintes da tabela `Aluno` para `AlunoHistorico` utilizando `INSERT ... SELECT`.

Vantagens

- Excelente para migração de dados.
- Muito utilizado em ETL.
- Evita processamento na aplicação.

Boas práticas

- Especifique sempre as colunas.
- Utilize transações em cargas críticas.

Exercício

Copie todos os professores do Departamento de Computação para uma tabela de auditoria.

Parte 05 — 2.5 SELECT INTO

2.5 SELECT INTO

Conceito

`SELECT INTO` cria uma **nova tabela** a partir do resultado de uma consulta.

```
SELECT * INTO AlunoBackup  
FROM Aluno;
```

Aplicação

Ideal para cópias temporárias, auditoria, migração e ambientes de testes.

Exemplo

Antes de atualizar milhões de registros, cria-se uma cópia da tabela para recuperação rápida.

Resultado

```
Command(s) completed successfully.
```

Boas práticas

- Não substitui backup do banco.
- Crie índices após a cópia, se necessário.

Parte 06 — 2.6 INSERT ... EXEC

2.6 INSERT ... EXEC

Conceito

Insere em uma tabela o resultado retornado por uma Stored Procedure.

```
CREATE TABLE Resultado(Nome varchar(100), Total money);  
  
INSERT INTO Resultado  
EXEC dbo.spRelatorioFinanceiro;
```

Aplicação

Muito utilizado em sistemas legados e processos ETL que reutilizam procedures existentes.

Vantagens

- Reaproveita lógica de negócio.
- Evita duplicação de código.

Parte 07 — 2.7 INSERT utilizando CTE

2.7 INSERT utilizando CTE

Conceito

CTE é a sigla para **Common Table Expression** (Expressão de Tabela Comum). É um conjunto de resultados nomeado e temporário, definido por meio da cláusula **WITH**, que existe apenas durante a execução da instrução seguinte (**SELECT**, **INSERT**, **UPDATE**, **DELETE** ou **MERGE**).

Na prática, a CTE funciona como uma "consulta auxiliar com nome": em vez de escrever uma subconsulta aninhada e de difícil leitura, o desenvolvedor a declara antes da instrução principal e depois referencia esse nome como se fosse uma tabela comum.

```
WITH AlunosAtivos AS (  
    SELECT * FROM Aluno WHERE Ativo=1  
)  
INSERT INTO AlunoBackup  
SELECT * FROM AlunosAtivos;
```

No exemplo acima, **AlunosAtivos** é a CTE: ela filtra os alunos ativos e, na sequência, o **INSERT** consome esse resultado para popular a tabela **AlunoBackup**.

Sintaxe geral

```
WITH NomeDaCTE (Coluna1, Coluna2, ...) AS  
(  
    -- consulta que define a CTE  
    SELECT ...  
)  
-- instrução que utiliza a CTE  
SELECT / INSERT / UPDATE / DELETE / MERGE ...
```

- A lista de colunas entre parênteses após o nome é opcional — só é obrigatória quando há colunas calculadas sem alias ou nomes repetidos;
- A CTE deve ser imediatamente seguida pela instrução que a utiliza; não é possível declarar uma CTE e usá-la "mais adiante" no script;
- É possível declarar **múltiplas CTEs** na mesma cláusula **WITH**, separando-as por vírgula, inclusive fazendo uma referenciar a outra.

```
WITH AlunosAtivos AS (  
    SELECT * FROM Aluno WHERE Ativo = 1  
) ,  
AlunosAprovados AS (  
    SELECT * FROM AlunosAtivos WHERE MediaFinal >= 7  
)  
INSERT INTO AlunoAprovadoBackup  
SELECT * FROM AlunosAprovados;
```

Características

- **Escopo limitado:** existe apenas durante a execução da instrução imediatamente seguinte; depois disso deixa de existir, diferente de uma tabela temporária (**#Tabela**);
- **Não é materializada em disco:** o SQL Server normalmente a trata como parte do plano de execução da consulta principal, embutindo sua lógica;
- **Pode ser recursiva:** usando **UNION ALL** para referenciar a própria CTE, útil para hierarquias (ex.: árvore de categorias, organograma);
- **Melhora a legibilidade:** substitui subconsultas aninhadas complexas por blocos nomeados e sequenciais, facilitando a leitura de cima para baixo.

CTE x Subconsulta x Tabela Temporária x View

Recurso	Escopo	Persistência	Reutilização
CTE	Uma única instrução	Não persiste	Só dentro da mesma instrução
Subconsulta	Uma única instrução	Não persiste	Não reutilizável (precisa repetir)
Tabela Temporária (#Tabela)	Sessão/conexão	Persiste até o fim da sessão	Reutilizável em várias instruções
View	Banco de dados	Persiste como objeto do banco	Reutilizável por qualquer consulta

Aplicação

Projetos com consultas extensas, múltiplos JOINS e filtros complexos.

Benefícios

- Melhor legibilidade.
- Facilidade de manutenção.
- Evita repetição da mesma subconsulta várias vezes na instrução.
- Permite organizar a lógica em etapas nomeadas antes da inserção final.

Limitações

- Escopo restrito a uma única instrução (não pode ser reaproveitada depois);
- Não aceita índices próprios, por não ser uma tabela física;
- CTEs recursivas mal escritas podem gerar loops de processamento custosos.

Exercício

Utilizando uma CTE, selecione apenas os clientes com mais de 5 pedidos e insira-os na tabela ClienteVIP.

Parte 08 — 2.9 INSERT ... OUTPUT

2.9 INSERT ... OUTPUT

Conceito

Captura automaticamente valores inseridos, como chaves IDENTITY.

```
DECLARE @Ids TABLE(Id INT);

INSERT INTO Produto(Nome)
OUTPUT INSERTED.Id INTO @Ids
VALUES('Notebook');
```

Aplicação

Parte 09 — 2.10 BULK INSERT

2.10 BULK INSERT

Conceito

Contexto: fiscalização tributária

[illegible]

Tabela de staging que recebe a carga bruta:


```
CREATE TABLE staging.nfe_bulk (  
    nrchaveacesso CHAR(44) NOT NULL,  
    vltotalnota DECIMAL(16,2) NULL,  
    dtemissao DATE NULL,  
    stnfe CHAR(1) NULL,  
    sqnfe BIGINT NULL,  
    tpnfe INT NULL  
);
```

Carga do arquivo para a tabela `staging.nfe_bulk`:

```
BULK INSERT staging.nfe_bulk  
FROM 'C:\cargas\nfe_sample.csv'  
WITH (  
    FIELDTERMINATOR = ';',  
    ROWTERMINATOR = '\n',  
    FIRSTROW = 2,  
    CODEPAGE = '65001',  
    TABLOCK,  
    MAXERRORS = 0  
);
```

Depois de carregado em `staging`, os dados só são movidos para as tabelas fiscais definitivas (`fiscal.NFE`) após validação (`TRY_CONVERT`), conforme mostrado em [02.07-CTE-INSERT.md](#) e no [roteiro-insercao-massa.md](#). Essa separação em duas etapas é importante em auditoria fiscal: garante que o arquivo original enviado pelo contribuinte fique preservado em `staging`, permitindo reprocessamento e rastreabilidade caso alguma nota seja rejeitada.

Sintaxe completa

```
BULK INSERT [ nome_do_banco . ] [ schema . ] nome_da_tabela  
FROM 'caminho_do_arquivo'  
WITH (  
    [ FIRSTROW = numero_da_linha ],  
    [ LASTROW = numero_da_linha ],  
    [ FIELDTERMINATOR = 'delimitador_de_coluna' ],  
    [ ROWTERMINATOR = 'delimitador_de_linha' ],  
    [ CODEPAGE = 'pagina_de_codigo' ],  
    [ DATAFILETYPE = { 'char' | 'native' | 'widechar' | 'widenative' } ],  
    [ BATCHSIZE = numero_de_linhas ],  
    [ ROWS_PER_BATCH = numero_de_linhas ],  
    [ MAXERRORS = numero_maximo_de_erros ],  
    [ ERRORFILE = 'caminho_do_arquivo_de_erro' ],  
    [ KEEPNULLS ],  
    [ TABLOCK ],  
    [ CHECK_CONSTRAINTS ],
```

```
[ FIRE_TRIGGERS ]
);
```

Detalhamento dos parâmetros

Parâmetro	Função
FIRSTROW	Define a primeira linha do arquivo a ser importada. Usado para pular o cabeçalho (ex.: <code>FIRSTROW = 2</code> ignora a linha de nomes das colunas).
LASTROW	Define a última linha a ser importada; útil para carregar apenas uma faixa do arquivo.
FIELDTERMINATOR	Caractere que separa as colunas dentro de cada linha (<code>;</code> , <code>,</code> , <code>\t</code> , etc.). Nos exemplos fiscais deste material, usa-se <code>;</code> porque o <code>,</code> já é usado como separador decimal em alguns arquivos de origem.
ROWTERMINATOR	Caractere que indica o fim de cada linha/registro (<code>\n</code> , <code>\r\n</code>). Deve corresponder exatamente à quebra de linha usada no arquivo.
CODEPAGE	Define a codificação de caracteres do arquivo. <code>'65001'</code> corresponde a UTF-8, importante para arquivos fiscais com acentuação (nomes de contribuintes, endereços).
DATAFILETYPE	Indica o tipo de dado do arquivo (<code>char</code> para texto simples, <code>native/widenative</code> para arquivos gerados por <code>bcp</code> em formato nativo).
BATCHSIZE	Quantidade de linhas processadas por lote (cada lote é <i>commitado</i> separadamente). Reduz o uso do log de transações em cargas muito grandes.
ROWS_PER_BATCH	Estima ao otimizador quantas linhas existem no arquivo, ajudando a escolher um plano de execução mais eficiente (sem dividir a carga em lotes como o <code>BATCHSIZE</code>).
MAXERRORS	Número máximo de linhas com erro tolerado antes de a carga inteira falhar. <code>MAXERRORS = 0</code> interrompe a carga no primeiro erro — recomendado em auditoria fiscal, onde uma nota mal formatada não deve passar despercebida.
ERRORFILE	Caminho de um arquivo onde o SQL Server grava as linhas rejeitadas, permitindo auditar exatamente o que falhou na carga.
KEEPNULLS	Mantém como <code>NULL</code> os campos vazios do arquivo, em vez de aplicar o valor <code>DEFAULT</code> da coluna.
TABLOCK	Aplica um bloqueio de tabela durante a carga, o que reduz a concorrência mas aumenta significativamente a performance — recomendado em cargas noturnas de grandes lotes de NF-e.
CHECK_CONSTRAINTS	Faz o SQL Server validar as <code>CONSTRAINTS</code> da tabela durante a carga (por padrão, <code>BULK INSERT</code> as ignora, por isso o uso de tabelas de staging sem regras rígidas).
FIRE_TRIGGERS	Faz os <code>INSERT TRIGGERS</code> da tabela de destino serem disparados durante a carga (por padrão não são).

Tratamento de erros

Em um cenário de fiscalização, é comum envolver o **BULK INSERT** em um bloco **TRY/CATCH**, registrando falhas em uma tabela de auditoria:

```
BEGIN TRY
    BULK INSERT staging.nfe_bulk
    FROM 'C:\cargas\nfe_sample.csv'
    WITH (
        FIELDTERMINATOR = ';',
        ROWTERMINATOR   = '\n',
        FIRSTROW         = 2,
        CODEPAGE          = '65001',
        MAXERRORS         = 0,
        ERRORFILE         = 'C:\cargas\erros\nfe_sample_erros.log'
    );
END TRY
BEGIN CATCH
    INSERT INTO auditoria.load_errors (file_path, error_message, error_time)
    VALUES ('C:\cargas\nfe_sample.csv', ERROR_MESSAGE(), SYSUTCDATETIME());
    THROW;
END CATCH
```

Aplicação

Migração de sistemas, Data Warehouse, ETL, cargas periódicas e recepção de arquivos fiscais (NF-e, NFC-e, CT-e, EFD) enviados por contribuintes para apuração e cruzamento de dados.

Vantagens

- Alto desempenho.
- Milhões de registros em poucos minutos.
- Processamento feito pelo próprio motor do banco, sem ferramenta externa.

Boas práticas

- Validar encoding do arquivo (**CODEPAGE**).
- Carregar primeiro em uma tabela de **staging** com tipos textuais, e só depois converter/validar para as tabelas definitivas.
- Executar em transações quando apropriado.
- Usar **MAXERRORS = 0** em cargas fiscais, para não aceitar arquivos parcialmente corrompidos silenciosamente.
- Registrar erros de importação com **ERRORFILE** e/ou tabela de auditoria.
- Preservar o arquivo original e os dados brutos em **staging** para rastreabilidade e reprocessamento.

Exercício

A SEFAZ recebe diariamente um arquivo CSV com as notas fiscais emitidas pelos contribuintes do estado. Usando **staging.nfe_bulk** como tabela de destino:

1. Escreva o comando **BULK INSERT** para carregar o arquivo **nfe_sample.csv**, tratando corretamente cabeçalho, delimitador e codificação.
2. Adicione **MAXERRORS** e **ERRORFILE** para capturar notas com formato inválido.
3. Explique por que a carga é feita em duas etapas (**staging** → **fiscal.NFE**) em vez de inserir diretamente na tabela final.

Parte 10 — 2.11 OPENROWSET(BULK)

2.11 OPENROWSET(BULK)

Conceito

O **OPENROWSET(BULK)** permite acessar arquivos externos como se fossem uma tabela temporária, possibilitando consultas e inserções diretamente a partir desses arquivos.

Sintaxe

```
INSERT INTO Cliente
SELECT *
FROM OPENROWSET(
    BULK 'C:\Importacao\clientes.csv',
    FORMAT='CSV',
    FIRSTROW=2
) AS Dados;
```

Funcionamento

1. O SQL Server abre o arquivo.
2. Os registros são interpretados conforme o formato especificado.
3. O resultado é disponibilizado como uma tabela.
4. O **INSERT** grava os dados na tabela destino.

Aplicações

- Importações ocasionais.
- Leitura de arquivos CSV.
- ETL simplificado.
- Data Lake.

Vantagens

- Não requer SSIS.
- Fácil utilização.

Limitações

- Requer permissões de acesso ao arquivo.

- Poucos recursos de transformação.

Exercício

Importe um arquivo CSV contendo a relação de cursos.

Parte 11 — 2.12 Assistente de Importação do SSMS

2.12 Assistente de Importação do SSMS

Conceito

O SQL Server Management Studio oferece um assistente gráfico para importar dados sem programação.

Fontes suportadas

- Excel
- CSV
- Access
- Oracle
- SQL Server
- ODBC

Fluxo

1. Tasks
2. Import Data
3. Escolha da origem
4. Escolha do destino
5. Mapeamento de colunas
6. Execução

Aplicações

- Pequenos projetos.
- Importações administrativas.
- Migração inicial.

Vantagens

- Interface amigável.
- Não exige SQL.

Limitações

- Pouca automação.
- Poucas transformações.

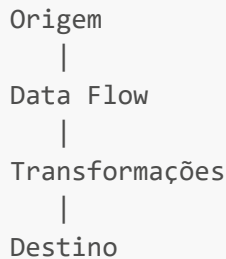
Parte 12 — 2.13 SQL Server Integration Services (SSIS)

2.13 SQL Server Integration Services (SSIS)

Conceito

O SSIS é a ferramenta oficial de ETL da Microsoft para extração, transformação e carga de dados.

Arquitetura



Componentes

- Data Flow
- Control Flow
- Lookup
- Derived Column
- Aggregate
- Conditional Split

Aplicações

- Data Warehouse.
- Business Intelligence.
- Integração entre sistemas.
- Migração de dados.

Exemplo

Importação diária de vendas das filiais para o banco corporativo.

Vantagens

- Alto desempenho.
- Grande quantidade de conectores.
- Agendamento pelo SQL Server Agent.

Parte 13 — 2.14 BCP (Bulk Copy Program)

2.14 BCP (Bulk Copy Program)

Conceito

O BCP é uma ferramenta de linha de comando para importação e exportação em alta velocidade.

Exemplo

```
bcp Escola.dbo.Aluno in alunos.csv -c -t, -S SERVIDOR -T
```

Aplicações

- Migração.
- Backup lógico.
- Cargas periódicas.
- Automatização via scripts.

Vantagens

- Muito rápido.
- Fácil automação.

Limitações

- Interface por linha de comando.
- Poucas transformações.

Parte 14 — 2.16 Linked Server

2.16 Linked Server

Objetivos

Ao final deste capítulo o aluno será capaz de:

- Compreender o conceito de Linked Server;
- Configurar a integração entre servidores SQL Server;
- Consultar e inserir dados remotos;
- Avaliar vantagens e limitações dessa abordagem.

Conceito

O **Linked Server** é um recurso do SQL Server que permite acessar objetos localizados em outro servidor de banco de dados como se fossem objetos locais.

Sua principal finalidade é integrar diferentes bases de dados sem necessidade de desenvolver rotinas de importação intermediárias.

Exemplo

```
INSERT INTO dbo.Cliente
SELECT *
FROM [SERVIDOR_MATRIZ].ERP.dbo.Cliente;
```

Aplicações

- Consolidação de bases corporativas;
- Migração entre servidores;
- Sistemas distribuídos;
- Integração entre matriz e filiais;
- Auditoria fiscal.

Vantagens

- Simplicidade de implementação;
- Não exige exportação para arquivos;
- Permite consultas distribuídas.

Limitações

- Dependência da rede;
- Impacto no desempenho em grandes consultas;
- Questões de segurança e autenticação.

Boas práticas

- Consultar apenas colunas necessárias;
- Evitar SELECT *;
- Utilizar filtros na origem;
- Monitorar tempo de resposta.

Exercício

Copie os produtos do servidor FILIAL01 para a tabela ProdutoBackup da matriz.

Parte 15 — 2.17 OPENQUERY

2.17 OPENQUERY

Conceito

O **OPENQUERY** executa uma consulta diretamente no **servidor remoto** e retorna apenas o resultado. Diferente do **OPENROWSET**, que abre uma conexão ad hoc a cada execução, o **OPENQUERY** sempre trabalha em cima de um **Linked Server** já configurado — é esse linked server (no exemplo, **OracleERP**) que informa ao SQL Server onde e como se conectar ao banco remoto.

Ou seja, antes de rodar o **OPENQUERY** do exemplo é necessário configurar o acesso ao servidor remoto.

Configurando o acesso ao servidor remoto

A configuração é feita uma única vez pelo DBA e envolve quatro passos: habilitar consultas distribuídas, criar o linked server com o provedor adequado, cadastrar as credenciais de acesso e liberar as opções necessárias para o OPENQUERY.

1. Habilitar consultas distribuídas ad hoc (se necessário)

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;

EXEC sp_configure 'Ad Hoc Distributed Queries', 1;
RECONFIGURE;
```

2. Criar o Linked Server

```
EXEC sp_addlinkedserver
    @server = 'OracleERP',
    @srvproduct = 'Oracle',
    @provider = 'OraOLEDB.Oracle',
    @datasrc = 'ERP_PROD';
```

- `@server` → nome que será usado nas consultas (é o mesmo nome referenciado dentro do `OPENQUERY`);
- `@srvproduct` → identificação do produto de origem;
- `@provider` → provider OLE DB instalado no servidor (ex.: `OraOLEDB.Oracle` para Oracle, `MSOLEDBSQL` para SQL Server);
- `@datasrc` → nome do serviço/instância de conexão configurado no cliente (ex.: entrada do `tnsnames.ora`).

3. Cadastrar as credenciais de acesso

```
EXEC sp_addlinkedsrvlogin
    @rmtsrvname = 'OracleERP',
    @useself = 'FALSE',
    @locallogin = NULL,
    @rmtuser = 'usuario_erp',
    @rmtpassword = 'senha_erp';
```

- `@useself = 'FALSE'` indica que a conexão remota não usará as credenciais do login local do SQL Server, e sim o usuário/senha informados em `@rmtuser/@rmtpassword`;
- `@locallogin = NULL` aplica essa credencial a qualquer login local que utilizar o linked server.

4. Permitir que o linked server execute OPENQUERY

```
EXEC sp_serveroption 'OracleERP', 'rpc out', 'true';  
EXEC sp_serveroption 'OracleERP', 'data access', 'true';
```

Com o linked server **OracleERP** criado e autenticado, o servidor remoto está pronto para ser consultado.

Utilizando o OPENQUERY

```
SELECT *  
FROM OPENQUERY  
(  
    OracleERP,  
    'SELECT CODIGO,NOME FROM CLIENTE '  
);
```

Também pode ser utilizado em inserções.

```
INSERT INTO Cliente  
SELECT *  
FROM OPENQUERY  
(  
    OracleERP,  
    'SELECT CODIGO,NOME FROM CLIENTE '  
);
```

A string entre aspas simples é enviada e processada **inteiramente pelo servidor remoto** (Oracle, no exemplo) — por isso deve seguir a sintaxe SQL do banco de origem, e não do SQL Server.

Quando utilizar

- Oracle → SQL Server
- PostgreSQL → SQL Server
- MySQL → SQL Server
- SQL Server → SQL Server

Vantagens

- Melhor desempenho que consultas distribuídas em muitos cenários;
- Processamento realizado no servidor remoto.

Limitações

- Exige a configuração prévia do Linked Server (provider, credenciais e opções de servidor);
- A string de consulta não é validada pelo SQL Server, apenas repassada ao servidor remoto;
- Erros de sintaxe só são identificados na execução, pois a consulta é uma string.

Exercício

Importe todos os fornecedores de um banco Oracle utilizando OPENQUERY.

Parte 16 — 2.18 Comparativo dos Métodos de Inserção

2.18 Comparativo dos Métodos de Inserção

Método	Complexidade	Volume	Principal aplicação
INSERT VALUES	Baixa	Baixo	Cadastros
INSERT múltiplas linhas	Baixa	Médio	Dados iniciais
INSERT SELECT	Média	Alto	Migração
SELECT INTO	Baixa	Médio	Backup temporário
INSERT EXEC	Média	Médio	Procedures
CTE + INSERT	Média	Médio	Consultas complexas
MERGE	Alta	Alto	Sincronização
INSERT OUTPUT	Média	Baixo	Recuperar IDENTITY
BULK INSERT	Média	Muito Alto	CSV
OPENROWSET	Média	Alto	Arquivos externos
Import Wizard	Baixa	Médio	Importação gráfica
SSIS	Alta	Muito Alto	ETL
BCP	Média	Muito Alto	Linha de comando
SqlBulkCopy	Alta	Muito Alto	Aplicações .NET
Linked Server	Média	Alto	Integração
OPENQUERY	Alta	Alto	Bancos heterogêneos

Recomendações

- OLTP → INSERT VALUES
- ETL → SSIS
- Grandes arquivos → BULK INSERT
- Aplicações C# → SqlBulkCopy
- Integração entre bancos → Linked Server ou OPENQUERY

Parte 17 — 2.20 Exercícios

2.20 Exercícios

Parte I

1. Insira cinco alunos utilizando INSERT VALUES.
2. Cadastre cinco cursos utilizando um único INSERT.
3. Copie todos os alunos para uma tabela Backup utilizando INSERT SELECT.
4. Crie uma nova tabela utilizando SELECT INTO.
5. Execute uma procedure utilizando INSERT EXEC.

Parte II

6. Utilize uma CTE para copiar apenas alunos ativos.
7. Sincronize duas tabelas utilizando MERGE.
8. Recupere o ID gerado utilizando INSERT OUTPUT.
9. Elabore um script BULK INSERT para importar um arquivo CSV.
10. Explique a diferença entre Linked Server e OPENQUERY.

Desafio

Desenvolva um processo completo de integração contendo:

- BULK INSERT
- MERGE
- INSERT OUTPUT
- Auditoria
- Tratamento de erros
- Transação

Apresente um diagrama do fluxo e justifique a escolha de cada método.

Parte 18 — Roteiro de Exercícios — Inserção em Massa (Multi-row, BULK)

Roteiro de Exercícios — Inserção em Massa (Multi-row, BULK)

Objetivo: praticar técnicas de inserção em massa (multi-row INSERT, INSERT...SELECT, BULK INSERT, OPENROWSET/BCP/SqlBulkCopy) aplicadas a dados fiscais (EFD, NF-e), com foco em desempenho, integridade e auditoria.

Exercício 1 — INSERT de múltiplas linhas (prático rápido)

1. Crie uma tabela `fiscal.NFE_TEST` com colunas: `sqnfe BIGINT`, `tpnfe INT`, `nrchaveacesso CHAR(44)`, `vltotalnota DECIMAL(16,2)`, `dtemissao DATE`, `stnfe CHAR(1)`.
2. Insira 5 notas via um único `INSERT` multi-row.

3. Verifique a inserção e documente o plano de execução (EXPLAIN/SHOWPLAN).

Exercício 2 — INSERT ... SELECT para transformar dados

1. Considere uma tabela `staging.raw_nfe` com campos textuais.
2. Realize um `INSERT INTO fiscal.NFE` usando `INSERT ... SELECT` com conversões (`TRY_CAST`/`TRY_CONVERT`) e filtros.
3. Registre as linhas rejeitadas em uma tabela `staging.nfe_rejeitadas`.

Exercício 3 — CTE + INSERT em massa

1. Use um `WITH cte AS (...)` para pré-processar dados (normalizar campos, remover duplicatas) e insira os resultados em `fiscal.NFE`.
2. Mostre o script e explique porque a CTE ajuda na legibilidade e modularidade.

Exercício 4 — BULK INSERT (T-SQL)

1. Prepare um CSV `nfe_sample.csv` com colunas: `nrchaveacesso;vltotalnota;dtemissao;stnfe;sqnfe;tpnfe` (ponto decimal `.` ou vírgula conforme configuração).
2. Crie uma tabela de staging `staging.nfe_bulk` com tipos textuais.
3. Use `BULK INSERT staging.nfe_bulk FROM 'C:\cargas\nfe_sample.csv' WITH (FIELDTERMINATOR=';', ROWTERMINATOR='\n', FIRSTROW=2, CODEPAGE='65001')`.
4. Valide e mova os registros para `fiscal.NFE` com `TRY_CONVERT`.

Exercício 5 — OPENROWSET(BULK) e PolyBase (se disponível)

1. Mostre como usar `OPENROWSET(BULK...)` para ler CSVs sem criar arquivo físico de staging (ex.: diretório de ingestão).
2. Se PolyBase estiver disponível, descreva os passos para criar external table e consultar diretamente.

Exercício 6 — BCP (linha de comando) e SqlBulkCopy (C#) — integração

1. Forneça comando `bcp` para importar o CSV diretamente para uma tabela staging.
2. Inclua um exemplo mínimo de uso de `SqlBulkCopy` em C# para enviar um `DataTable` ao SQL Server.

Exercício 7 — Estratégia de carga para grandes volumes (procedimento)

1. Escreva um roteiro de passos para ingestão de 10 milhões de linhas fiscais:
 - preparar CSVs particionados por mês;
 - carregar em `staging` com `BULK INSERT`/BCP;
 - desabilitar índices e constraints durante carga (quando seguro);
 - executar validação e conversão por lote;
 - inserir em tabelas finais e recriar índices;
 - coletar estatísticas e executar `UPDATE STATISTICS`.
2. Explique riscos e rollback strategies.

Exercício 8 — Tratamento de erros e auditoria

1. Implemente TRY/CATCH numa procedure que executa o fluxo de carga e grava erro em `auditoria.load_errors` com detalhes do arquivo/linha/mensagem.
2. Mostre um exemplo de registro de erro e como reprocessar apenas linhas com falha.

Exercício 9 — Fluxo completo de staging → validação → tabela final

1. Use os arquivos CSV de exemplo disponíveis em `exemplos/` para praticar um fluxo completo de ingestão:
 - `staging_nfe_20.csv`
 - `staging_item_nfe_20.csv`
 - `staging_nfce_20.csv`
 - `staging_item_nfce_20.csv`
 - `staging_cte_20.csv`
 - `staging_efd_registro_20.csv`
2. Crie as tabelas de staging sugeridas no script `exemplos/staging_bulk_tabelas.sql`.
3. Faça o **BULK INSERT** de cada CSV para a respectiva tabela de staging, usando `FIELDTERMINATOR=';', ROWTERMINATOR='\n', FIRSTROW=2` e `CODEPAGE='65001'`.
4. Valide os dados carregados em staging com consultas simples, por exemplo:
 - verificar quantidade de linhas por arquivo;
 - identificar valores nulos ou inconsistentes;
 - confirmar se as datas estão no formato esperado.
5. Mova os registros para as tabelas finais com `INSERT ... SELECT` e conversões explícitas:
 - `TRY_CONVERT(DATE, data_emissao)` para datas;
 - `TRY_CONVERT(DECIMAL(15,2), valor_total)` para valores monetários;
 - `TRY_CONVERT(BIGINT, id_nfe)` ou `TRY_CONVERT(BIGINT, id_nfce)` quando necessário.
6. Registre linhas inválidas em uma tabela de rejeição, por exemplo `staging.rejeicoes_bulk`, com colunas: `origem, motivo, dados_brutos`.
7. Compare o número de linhas carregadas em staging com o número efetivamente inserido nas tabelas finais e explique diferenças.
8. Discuta por que o uso de staging melhora a rastreabilidade, a auditoria e a recuperação de falhas em processos de carga.

Entregáveis

- Scripts SQL: `CREATE TABLE`, `INSERT` (multi-row), `BULK INSERT`, `INSERT ... SELECT`, CTE scripts.
- Arquivo CSV de exemplo: `nfe_sample.csv`.
- Comando `bcp` e snippet `SqlBulkCopy`.
- Relatório curto (1 página) com justificativas de cada escolha (tipos, delimitador, handling de encoding, uso de índices).

Parte 19 — Respostas e Explicações — Inserção em Massa (Multi-row, BULK)

Respostas e Explicações — Inserção em Massa (Multi-row, BULK)

Este documento responde passo a passo ao roteiro de exercícios, explicando decisões técnicas e fornecendo exemplos SQL e comandos.

Exercício 1 — INSERT de múltiplas linhas

Solução (exemplo)

[illegible]

Explicações

- Usamos `CHAR(44)` para chave de acesso porque o tamanho é fixo; isso economiza espaço e melhora indexação.
- Multi-row `INSERT` reduz round-trips ao servidor comparado a múltiplos `INSERT` individuais.
- Verificar plano de execução (`SET SHOWPLAN_TEXT ON / EXPLAIN`) ajuda a confirmar se o DB usa abordagem eficiente.

Exercício 2 — INSERT ... SELECT

Solução (exemplo)

```
-- tabela staging
CREATE TABLE staging.raw_nfe (
  raw_id BIGINT IDENTITY(1,1) PRIMARY KEY,
  nrchave NVARCHAR(100),
  vltotal NVARCHAR(50),
  dtemissao NVARCHAR(50),
  stnfe NVARCHAR(5),
  imported_dt DATETIME2 DEFAULT SYSUTCDATETIME()
);
```

```

-- mover validos
INSERT INTO fiscal.NFE (sqnfe, tpnfe, nrchaveacesso, vltotalnota, dtemissao,
stnfe)
SELECT NEXT VALUE FOR seq_nfe, 1,
       LEFT(r.nrchave,44) AS nrchaveacesso,
       TRY_CAST(REPLACE(r.vltotal,',','.') AS DECIMAL(16,2)) AS vltotalnota,
       TRY_CAST(r.dtemissao AS DATE) AS dtemissao,
       LEFT(r.stnfe,1) AS stnfe
FROM staging.raw_nfe r
WHERE LEN(r.nrchave) >= 44
      AND TRY_CAST(REPLACE(r.vltotal,',','.') AS DECIMAL(16,2)) IS NOT NULL
      AND TRY_CAST(r.dtemissao AS DATE) IS NOT NULL
      AND NOT EXISTS (SELECT 1 FROM fiscal.NFE n WHERE n.nrchaveacesso =
LEFT(r.nrchave,44));

-- rejeitados
INSERT INTO staging.nfe_rejeitadas (raw_id, motivo)
SELECT raw_id, 'Formato invalido' FROM staging.raw_nfe r
WHERE NOT (LEN(r.nrchave) >= 44 AND TRY_CAST(REPLACE(r.vltotal,',','.') AS
DECIMAL(16,2)) IS NOT NULL);

```

Explicação

- `TRY_CAST` e `REPLACE` ajudam a converter formatos diferentes (vírgula/ponto decimal).
- `INSERT ... SELECT` é eficiente pois a conversão é feita no servidor com set-based operations.
- Registrar rejeitados permite reprocessamento sem perder a linha original.

Exercício 3 — CTE + INSERT

Solução (exemplo)

```

WITH cleaned AS (
  SELECT DISTINCT
    LEFT(r.nrchave,44) AS nrchaveacesso,
    TRY_CAST(REPLACE(r.vltotal,',','.') AS DECIMAL(16,2)) AS vltotalnota,
    TRY_CAST(r.dtemissao AS DATE) AS dtemissao,
    LEFT(r.stnfe,1) AS stnfe
  FROM staging.raw_nfe r
  WHERE LEN(r.nrchave) >= 44
)
INSERT INTO fiscal.NFE (sqnfe, tpnfe, nrchaveacesso, vltotalnota, dtemissao,
stnfe)
SELECT NEXT VALUE FOR seq_nfe, 1, nrchaveacesso, vltotalnota, dtemissao, stnfe
FROM cleaned c
WHERE vltotalnota IS NOT NULL;

```

Explicação

- CTE melhora legibilidade e separa a transformação da inserção.
- **DISTINCT** evita duplicatas no staging; decisões como **DISTINCT** devem ser ponderadas conforme requisitos.

Exercício 4 — BULK INSERT (T-SQL)

Exemplo de CSV (nfe_sample.csv)

[illegible]

Script de criação da tabela staging

```
CREATE TABLE staging.nfe_bulk (
  nrchaveacesso CHAR(44) primary key,
  vltotalnota DECIMAL(16,2),
  dtemissao DATE,
  stnfe CHAR(1),
  sqnfe BIGINT,
  tpnfe INT
);
```

Comando BULK INSERT

```
BULK INSERT staging.nfe_bulk
FROM 'C:\cargas\nfe_sample.csv'
WITH (
    FIELDTERMINATOR=';',
    ROWTERMINATOR='\n',
    FIRSTROW=2,
    CODEPAGE='65001',
    MAXERRORS=0
);
```

Explicações

- `CODEPAGE='65001'` garante UTF-8; ajuste conforme origem do CSV.
- `FIRSTROW=2` pula cabeçalho.
- Depois de carregar em `staging`, utilizar `INSERT ... SELECT` com validações.

Exercício 5 — OPENROWSET(BULK) e PolyBase

OPENROWSET exemplo

```
SELECT *
FROM OPENROWSET(
    BULK 'C:\cargas\nfe_sample.csv',
    FORMAT='CSV',
    PARSER_VERSION='2.0'
) AS t;
```

Observações

- `OPENROWSET(BULK...)` permite leitura ad-hoc sem objeto persistente.
- PolyBase oferece external tables e é recomendado para grandes volumes e integração com HDFS/ADLS.

Exercício 6 — BCP e SqlBulkCopy

Comando BCP (exemplo)

```
bcpx "staging.nfe_bulk" in "C:\cargas\nfe_sample.csv" -S myserver -d SEFAZ_FISCAL -
T -c -t";" -r"\n"
```

SqlBulkCopy (C# minimal)

```
using (var bulk = new SqlBulkCopy(connString)) {
    bulk.DestinationTableName = "staging.nfe_bulk";
    bulk.WriteToServer(dataTable);
}
```

Explicação

- `bcpx` é leve e rápido para cargas; `SqlBulkCopy` é programático e permite transformação em memória antes da carga.

Exercício 7 — Estratégia de carga para 10M linhas

Roteiro resumido

1. Particionar fonte em arquivos por mês (por exemplo `nfe_2026_07.csv`).
2. Para cada arquivo:
 - `BULK INSERT` para `staging` (sem índices);
 - Validar e transformar em lotes (10k-100k) via `INSERT ... SELECT` transacional;
 - Recriar índices e atualizar estatísticas;

3. Monitorar `tempdb` e I/O; usar `TABLOCK` em cargas massivas para melhorar throughput.

Riscos/mitigações

- Risco: locks longos — mitigar com batches e menor transaction scope.
- Risco: consumo de log — usar recuperação em massa e planejar backups.

Exercício 8 — Tratamento de erros e auditoria

Procedure de exemplo (esqueleto)

```
CREATE PROCEDURE staging.LoadNfeFile @FilePath NVARCHAR(4000)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        BULK INSERT staging.nfe_bulk FROM @FilePath WITH (FIELDTERMINATOR=';',
FIRSTROW=2);
        -- validação e movimentação
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        INSERT INTO auditoria.load_errors (file_path, error_message, error_time)
        VALUES (@FilePath, ERROR_MESSAGE(), SYSUTCDATETIME());
        THROW;
    END CATCH
END
```

Explicação

- Registrar erros com `ERROR_MESSAGE()` facilita reprocessamento e auditoria.
- Transações garantem atomicidade por arquivo/ lote.

Observações finais e justificativas de design

- Use `DECIMAL` para valores financeiros; evitar `FLOAT`.
- Prefira CSV com delimitador `;` em ambientes que usam `,` como decimal separator; documente e normalize antes da carga.
- Crie índices apenas após carga inicial para evitar overhead em inserções massivas.
- Sempre manter um `staging` imutável (raw) para poder reprocessar sem perda de origem.

Arquivos relacionados:

- `roteiro-insercao-massa.md` (enunciado)
- `nfe_sample.csv` (exemplo de carga)

- [insercao-exemplos.sql](#) (coleção de exemplos SQL)